UNIVERSIDAD ANDRÉS BELLO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN E INFORMÁTICA

PARADIGMAS DE PROGRAMACIÓN

TAREA 1

Sistema de Gestión de Rutinas de Entrenamiento Personalizadas

Integrante:

- Francisco Molinet Henriquez
 -RUT: 20.441.917-5

NRC: 8025

Ciudad: Viña del Mar

Fecha: 16 de Abril de 2026

Introducción

Motivación

En los centros deportivos de hoy en día, los preparadores físicos enfrentan el desafío de crear planes de entrenamiento personalizados para cada cliente teniendo en cuenta sus objetivos físicos, el nivel de condición y disponibilidad de tiempo disponible. Crear estos planes a mano puede resultar muy ineficiente y propensa a errores, especialmente al intentar respetar restricciones como no repetir ejercicios en semanas consecutivas.

Objetivo del trabajo

Desarrollar una solución orientada a objetos en C++ que facilite gestionar ejercicios y generar rutinas personalizadas, aplicando los conceptos de:

- Abstracción: Modelar las entidades significativas del problema.
- Encapsulamiento: Resguardar la información y ofrecer interfaces controladas.
- Herencia: Reutilizar código entre tipos de ejercicios
- Polimorfismo: Manejar objetos de varias clases de manera uniforme.

Estructura del documento

El documento se divide en cuatro secciones fundamentales: Descripción de la solución propuesta con el diagrama de clases, descripción de las clases desarrolladas, justificación del diseño y conclusión del trabajo realizado.

Descripción de la solución

Modelo propuesto

El sistema está compuesto por 4 clases principales:

Clase	Responsabilidad	Relación
Ejercicio	Clase base que define atributos comunes	Base de la jerarquia
EjercicioCardio	Especialización para ejercicios cardiovasculares	Hereda de ejercicio
EjercicioFuerza	Especialización para ejercicios de fuerza	Hereda de ejercicio
Rutina	Agrupa ejercicios para un cliente en una semana específica	Agregación de ejercicio

Justificación de diseño

Herencia
Las clases EjercicioCardio y EjercicioFuerza heredan públicamente de Ejercicio. Esto se justifica porque:

- Ambos son tipos de Ejercicio: Relación es-un
- Comparten atributos comunes: código, nombre, tipo, intensidad, tiempo, descripción y última semana de uso.
- Permite reutilizar código de la clase base y poder especializar comportamientos.

Polimorfismo
El método mostrarInfo() se declara virtual en la clase base y se sobrescribe con override en cada clase hija.

- En Ejercicio.h
  -virtual void mostrarInfo();

- En EjercicioCardio.cpp
  -void EjercicioCardio::mostrarInfo() override {
  Ejercicio::mostrarInfo();
  cout << "Implemento: " << implemento << endl;
}

Composición: Clase Rutina
La clase Rutina contiene un vector<Ejercicio*>, creando de esta manera una relación de composición donde una rutina puede estar compuesta de múltiples ejercicios y los ejercicios existen independientemente de la rutina.

Encapsulamiento: Atributos privados
Los atributos son private, los cuales solo se puede acceder mediante métodos get y set. Esto permite proteger la integridad de los datos.

Diagrama de clases

Detalles de clases implementadas

1. Clase Ejercicio (Base)

Archivos: Ejercicio.h, Ejercicio.cpp

Finalidad: Definir la interfaz común y atributos compartidos por todos los ejercicios.

Métodos principales:
- Constructor con lista para inicializar atributos
- Destructor virtual
- mostrarInfo() virtual
- Métodos get y set para todos los atributos

2. Clase EjercicioCardio (Derivada)

Archivos: EjercicioCardio.h, EjercicioCardio.cpp

Finalidad: Especializar ejercicios de cardio con su atributo propio.

Atributos propios: implemento(string)

Métodos principales:
- Constructor que llama a clase padre con la lista de inicialización.
- mostrarInfo() override para mostrar la información junto con el implemento.

3. Clase EjercicioFuerza (Derivada)

Archivos: EjercicioFuerza.h, EjercicioFuerza.cpp.

Finalidad: Especializar ejercicios de fuerza con sus propios atributos.

Atributos propios: repeticiones(string), series(int), peso(double)

Métodos principales:
- Constructor que llama a clase padre con la lista de inicialización.
- mostrarInfo() override para mostrar series, repeticiones y peso.
- Métodos get y set específicos.

4. Clase Rutina

Archivos: Rutina.h, Rutina.cpp

Finalidad: Agrupar ejercicios para un cliente en específico en una semana determinada.

Atributos: nombreCliente(string), numeroSemana(int), ejercicios (vector<Ejercicio*>).

Métodos principales:

- agregarEjercicio(Ejercicio*, int) para validar restricción en semanas consecutivas y agregar a la lista vector.
- calcularTiempoTotal() para sumar el tiempo de todos los ejercicios agregados.
- mostrarRutina() muestra la información completa de la rutina.

Conclusión

1. Cumplimiento de objetivos

Objetivo general: El sistema facilita crear, listar, actualizar y eliminar ejercicios, además de generar rutinas personalizadas según las normas del negocio. Los componentes fueron implementados utilizando los pilares de POO.

Aplicación de paradigmas:

- La jerarquía en las clases permite reutilizar código y poder extender funcionalidades.
- Encapsulamiento: Protege los atributos mediante private y expone acceso el cual se puede controlar mediante métodos get y set.
- Polimorfismo: A través de virtual/override permite comportamiento dinámico según el tipo de objeto.

2. Aprendizajes obtenidos

- **Herencia y constructores**: Me familiarice con el uso de listas de inicialización para poder llamar constructores de la clase base.
- **Polimorfismo**: Entendí de qué manera un puntero base puede hacer referencia a objetos derivados y ejecutar métodos que han sido sobreescritos.
- **Punteros y su gestión:** Ejercite la creación y liberación de memoria dinámica, lo que ayudó a evitar fugas de memoria.
- **Diseño de clases**: Se aplicó la división de responsabilidades, permitiendo así que el código sea más fácil de entender y mantener. Cada entidad tiene su función específica y no se mezclan responsabilidades.